3. Perform tokenization using BERT and GPT tokenizers and compare their outputs.

Aim:

To perform tokenization using BERT and GPT tokenizers and compare their outputs.

Algorithm:

Start the program.

Import `AutoTokenizer` from the Transformers library.

Load the BERT tokenizer using `bert-base-uncased`.

Load the GPT-2 tokenizer using `gpt2`.

Define a sample input sentence.

Tokenize the sentence using the BERT tokenizer.

Tokenize the same sentence using the GPT-2 tokenizer.

Convert the tokens into numerical token IDs using both tokenizers.

Display the tokens and token IDs generated by BERT and GPT-2.

Compare the token counts and tokenization outputs.

Stop the program.

Code:

```
import torch

text = "Artificial intelligence is changing the world."

# Convert text into model inputs
inputs = tokenizer(
    text,
    return_tensors="pt"
)

# Generate embeddings without calculating gradients
model.eval()

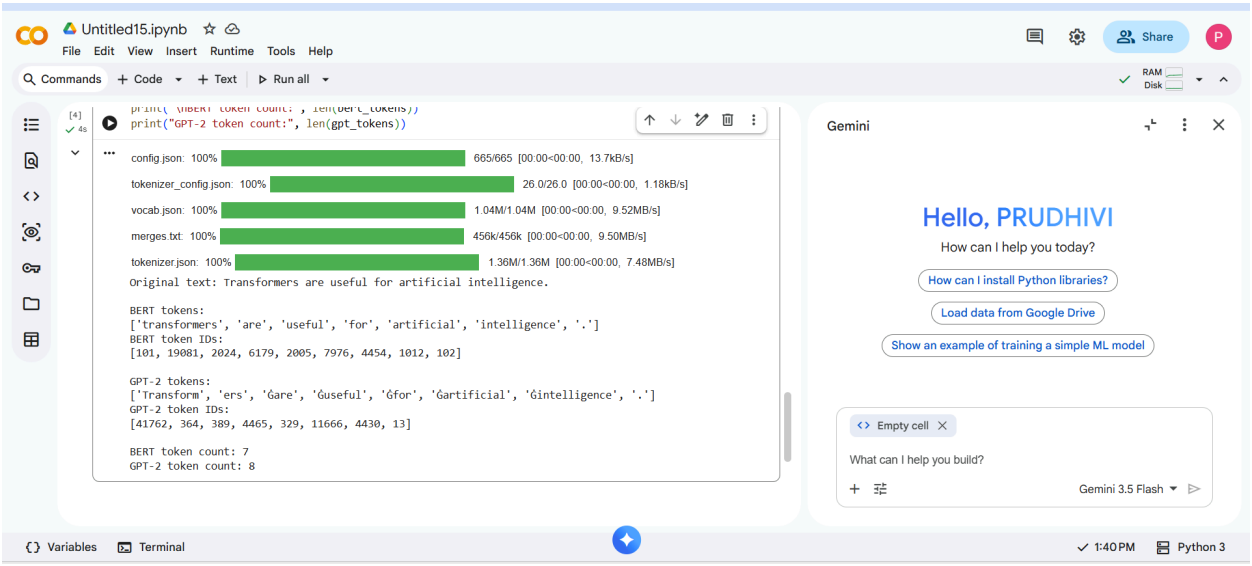
with torch.no_grad():
    outputs = model(**inputs)

# Contextual embeddings for every token
embeddings = outputs.last_hidden_state

print("Input text:", text)
print("Input token IDs:", inputs["input_ids"])
print("Embedding shape:", embeddings.shape)

# Display the first token's embedding
print("\nFirst token embedding:")
print(embeddings[0, 0, :10])
```

Screenshot:



Result:

BERT generated numerical contextual embeddings for the input tokens, representing their learned contextual information.